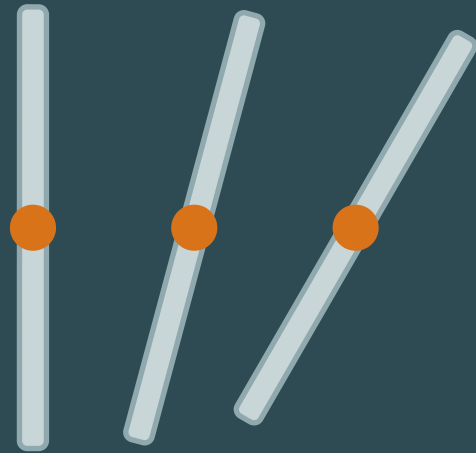
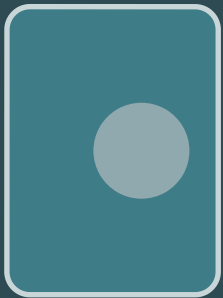


VSX3000 Performance Testing: Test Procedure and Robot Program

Running the five capture series and what the robot program must do



What we are measuring, and why



The five series

- A: depth noise against distance, incidence angle and drift, on the board
- B: lateral resolution from the edge targets, at two step heights
- Z: the smallest detectable depth step and the depth quantum
- C: true against sensed area of disks and cutouts
- D: the minimum detectable feature size, as a detection probability



What the robot gives us

- Commanded, repeatable, logged target poses in front of a fixed sensor
- Ground truth in the sensor frame through one registration from the board's plane
- Read-back poses at 0.01 mm: the step truth of series Z
- Every pose approached from the same direction, so backlash enters no comparison



What the session produces

- One session folder of raw .mc frames with a manifest that says, for every file, which target was where
- Six analyses that write `forward_model_parameters.json`, the hand-off to the forward-noise simulator
- Analysis E, boundary bias, reuses the B and C frames and needs no captures of its own



If the manifest is wrong, the results are wrong: much of the procedure is about getting that table right.

One session folder, 7,202 poses, 42,920 frames



7,202

robot poses in the default plan, in the planner's order



7.2 h

of robot time at 10 frames per second and 3 s per move; two to three working days with swaps, checks and warm-up



430 GB

of .mc frames at 10 GB per 1,000 frames; check storage before starting



Save every frame raw, with one manifest row per frame, so any analysis can be re-run without re-capturing.

The default plan is the capture budget of procedure section 13. Outside it are the optional captures the engineer plans only when needed: the filters-off repeat of A and B, the staircase of Z, the lateral sweep of B, the open-background variant of C, and the separate 8 hour drift run taken with the robot idle.

The session at a glance



Preparation before the first session



Robot

- Flange confirmed against ISO 9409-1-50-4-M6; repeatability 0.05 mm or better
- Controller reports the actual pose at 0.01 mm with a time stamp
- Each fixture weighed; tool load data entered
- Robot program written to the pose-list interface; dry run of every pose at reduced speed with the largest target
- Safety review



Sensor and computers

- Capture software set for the trigger and the pose-id file names
- Production sensor configuration chosen and recorded in sensor_config.json; filters-off repeat decided
- At least 430 GB free on the capture computer
- This repository installed on the analysis computer and self-tested



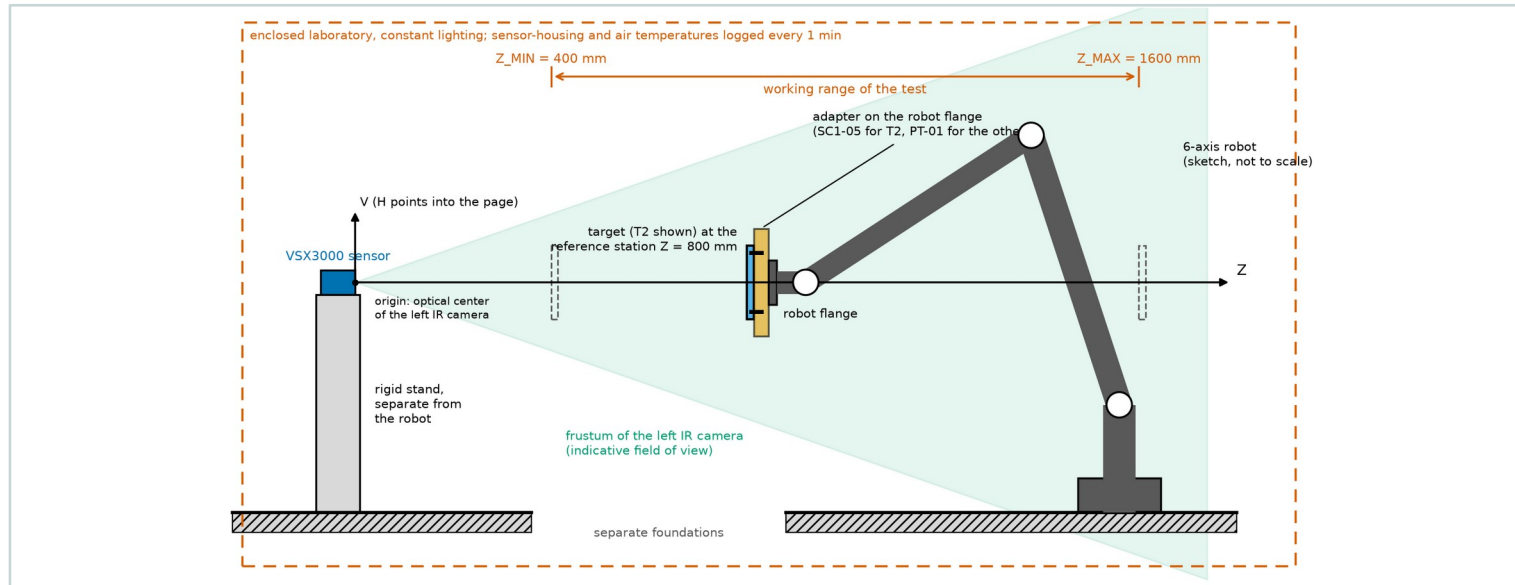
Cell

- Layout study: target centers from 400 to 1600 mm, the five field positions and tilts to 45 degrees inside the robot's reach, before the sensor mount is fixed
- Run-out base plate bolted to the table
- Robot base frame identified and recorded; temperature loggers started and their clocks checked



The targets, the adapter and the tools themselves are in the procurement and build deck and in section 1 of the procedure.

The sensor stays fixed; the robot carries the target



- ✓ The sensor stands on its own rigid mount, mechanically separate from the robot base; a bump means a new session
- ✓ The robot carries the target on the adapter anywhere between 400 and 1600 mm, the working range of the test
- ✓ The laboratory is enclosed and its lighting constant: no light meter, no ambient-light log, no changes during the session
- ✓ Sensor-housing and air temperatures are logged every minute into `environment_log.csv`
- ✓ The origin of every pose is the optical center of the left IR camera; H and V are the image axes, Z the optical axis

Configuration, warm-up, settle and frame checks



Sensor configuration (engineer)

step 4.2

Auto-exposure off; exposure, gain, emitter power and trigger mode fixed; every filter setting, the SDK and firmware versions in sensor_config.json under a config_id



Warm-up

step 4.3

Power for at least 45 minutes; then the board at 800 mm, 10 frames every minute, until the mean plane Z drifts less than 0.1 of its temporal noise over 10 minutes



Settle and vibration

step 4.4

At 400 mm, 100 frames with the servos on after a move and 100 with the brakes on; raise the 2 s settle time or stiffen the mount if the servo-on noise exceeds the brakes-on noise by more than 10 percent



Intrinsics and frame checks (engineer)

step 4.5

Dagger values from the SDK into sensor_config.json; valid depth at 400 and 1600 mm; depth registered to the left IR image within 0.5 px; baseline direction from the occlusion band

Procedure section 3. Whatever settle time passes the check is the one the robot program uses; write the date, serial numbers, versions, base frame and plate reports in session_log.md.

Registration: 30 hand-jogged poses of the board

1 Mount

The board on its adapter;
mount check



2 Jog

To each of the 30
registration poses of the
plan: 400 to 1600 mm,
across the field, tilted
about H and V up to plus
or minus 20 degrees, the
board fully visible



3 Capture

10 depth frames and the
read-back robot pose at
each; files named by the
rule of section 5



4 Solve (engineer)

Fit the plane of each
capture; register --
observations
observations.csv --out
registration.json

0.15 mm

RMS plane-distance
residual to accept

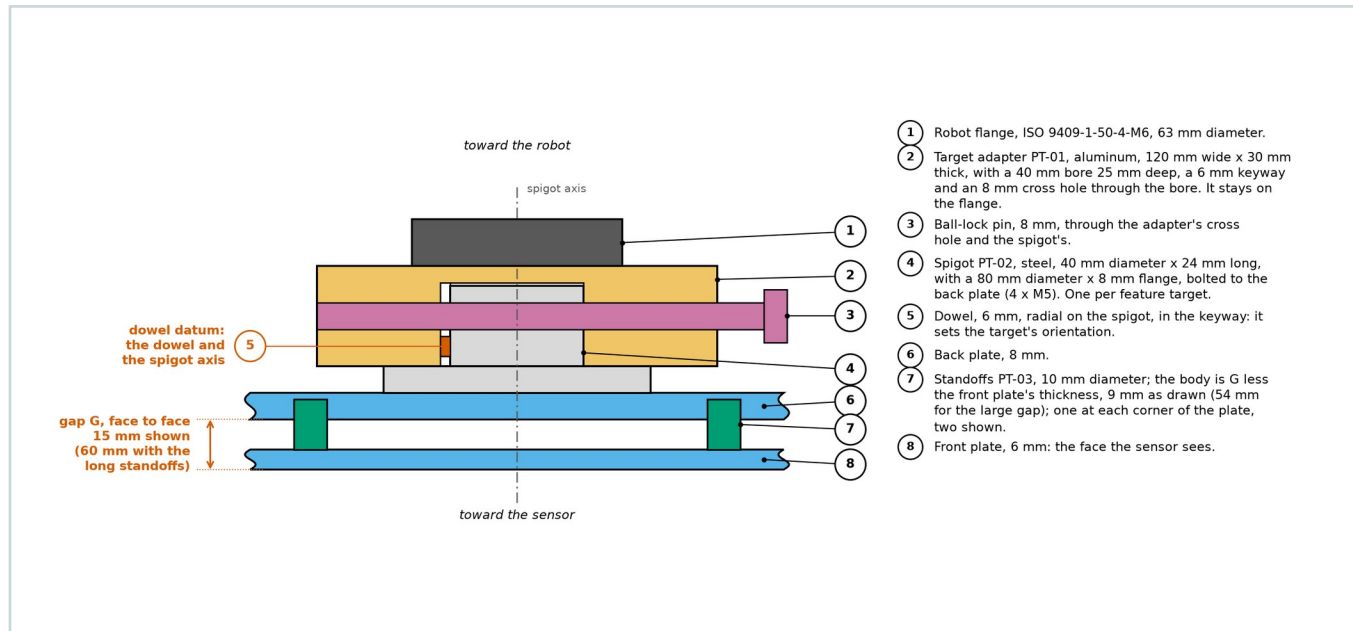
~0.1 mm

expected standard error of
the camera's Z offset,
recorded with the result



If the residual is larger, add poses with larger
tilts or check the mount, then solve again.
**Registration comes first, so the poses are jogged
by hand; the exact pose is solved afterward.**

The mount check after every target swap



Move the mounted target to the mount-check station, 800 mm, centered and fronto-parallel; capture a few frames into a folder of their own



check_captures --mount-check fits the front plane and compares it with the registered pose: Z within 0.15 mm, tilt within 0.05 degrees



For a feature target it also locates one feature outline in the image and compares H and V with the as-built datum offsets: within 0.5 px

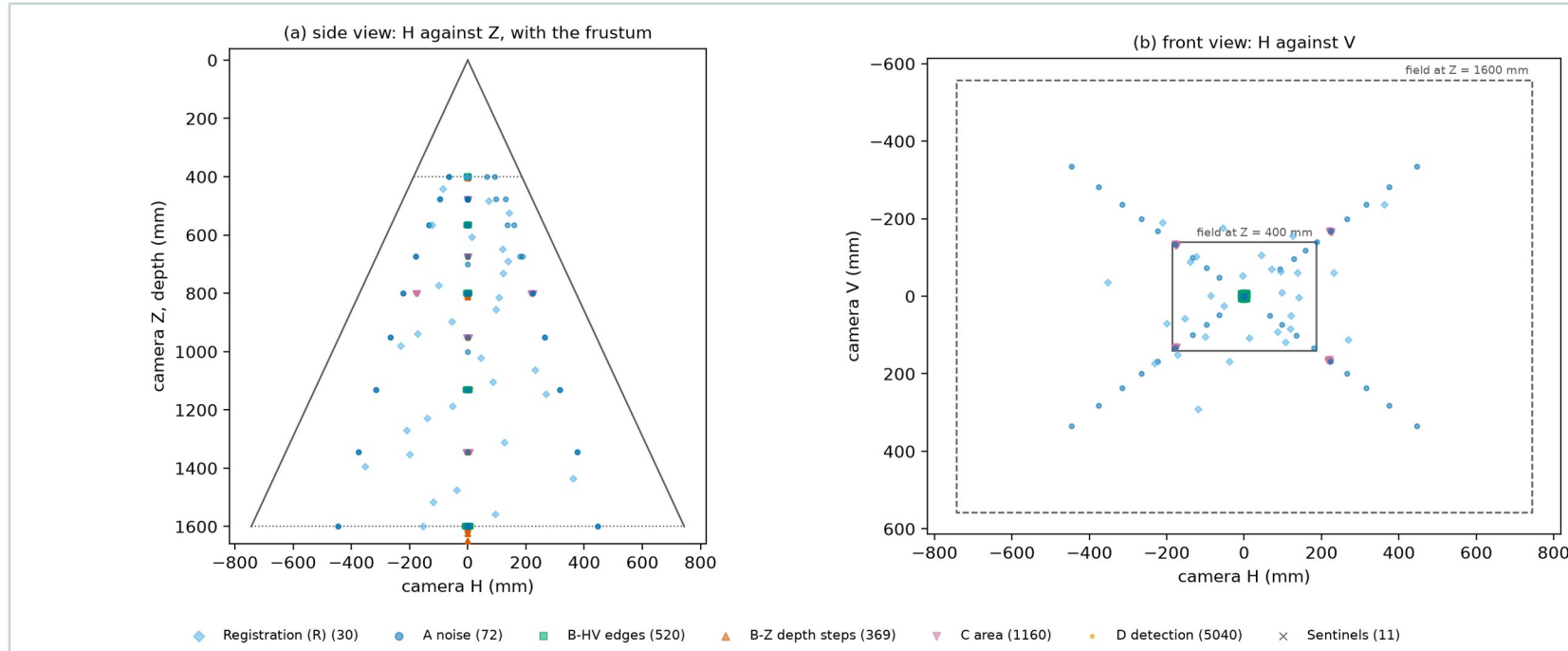


PASS or FAIL per check with the measured value; otherwise re-seat the target or correct the datum record



The spigot and dowel return every target to the same datum, so a passed check makes re-mounting safe

One plan, 9 stations, 5 field positions



The default plan: target centers in the sensor frame, side and front views, colored by series. Each cloud around a station is a set of random lateral offsets.

9

stations of one geometric ladder, 400 to 1600 mm at a ratio of 2 to the quarter power

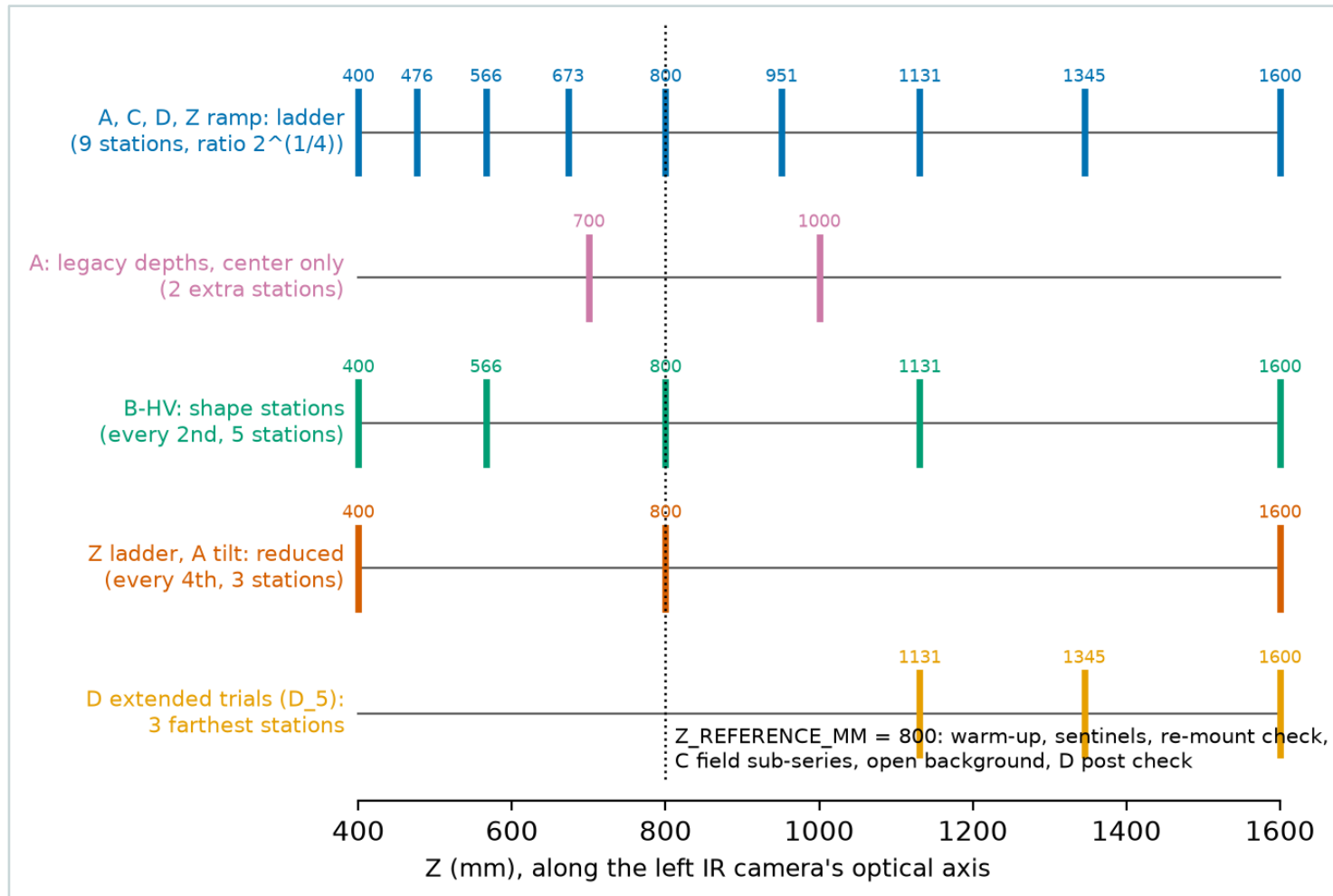
5

field positions: the center and four corners at 0.6 of the half field

1 seed

logged in session_log.md; the same seed and inputs give the same plan again

Which series visits which stations



All 9 stations: series A, C and D, and the ramp of Z; A adds the legacy depths 700 and 1000 mm at the center



Every second station, the 5 shape stations 400, 566, 800, 1131 and 1600 mm: series B



Every fourth station, the 3 reduced stations 400, 800 and 1600 mm: the step ladder of Z; the tilt sub-series of A at 476, 800 and 1600 mm



The 3 farthest stations: the extended detection trials of D



The reference station 800 mm: warm-up check, sentinels, mount check, field sub-series of C, open-background variant, lateral sweep

What the robot program must do



Inputs

- poses.csv from the planner: one row per pose in execution order, with the wanted target pose in the camera frame and the flange pose from the registration
- The settle time of step 4.4, 2 s unless the check raised it



Behavior

- Joint move to a point short of the pose along the approach, then a linear move onto it: from below in Z and from the negative side of H and V, every pose, every series
- Series Z: back off 2 mm toward smaller Z, then move up onto the pose
- Never change the base frame, the speed settings or the sensor configuration



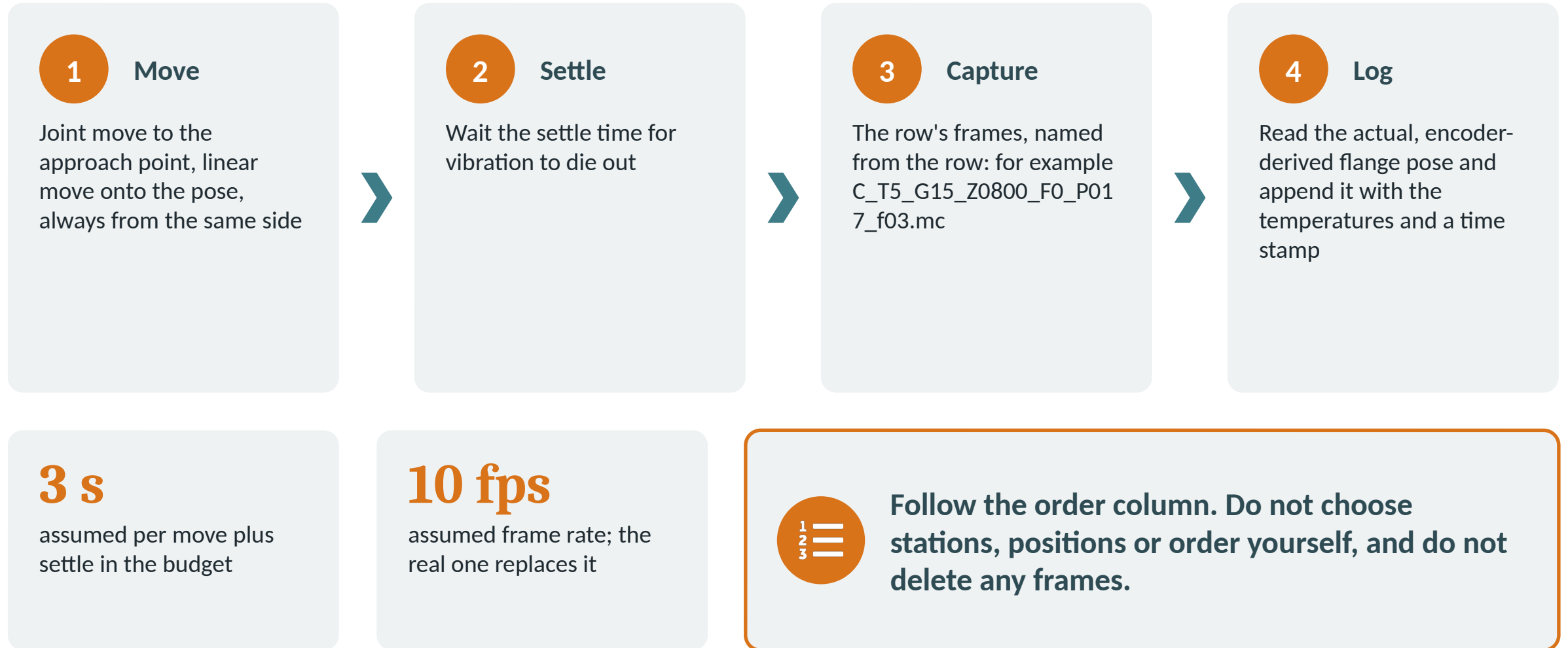
Outputs

- Trigger the row's frames through the capture software, named from the row by the rule of section 5 (procedure, target, gap, station, field, pose, frame; the example is on the next slide)
- Append the REPORTED flange pose, the temperatures and a time stamp to the pose log; note where it is written



Once the robot model and controller are settled, the robot program can be written with Claude Code.

The per-pose loop



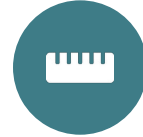
Series A and B: the board, then the edge targets



A, noise on the board

T2, 100 frames per pose

All 9 stations at the 5 field positions plus the legacy depths at the center, in randomized order; then the tilt sub-series at 476, 800 and 1600 mm (0, 15, 30, 45 degrees about V then H, 50 frames); then dismount, re-mount and repeat the 800 mm center station



B, edges

T3a then T3b, 30 frames per pose

At each of the 5 shape stations: 1 nominal pose and 25 jitter poses with logged random offsets up to plus or minus 4 px; the gap at 15 mm, then at 60 mm; the square's 5 degree slant is part of the target



Sentinels

30 frames at 800 mm

Captured on whatever target is mounted, before A, every 60 minutes, after the last pose of each series; never swap a target for one



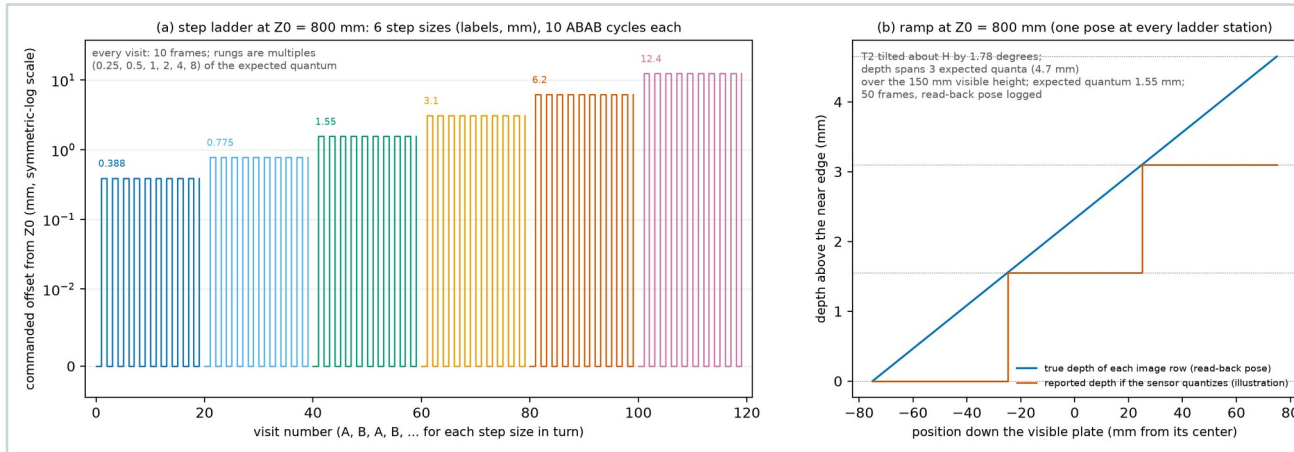
Optional

engineer decides

The filters-off repeat of A and B with a new config_id; the lateral sweep of T3a at 800 mm, 20 poses in H and 20 in V in 0.1 px steps, approached from alternating sides

Procedure sections 6 and 7. Each target stays mounted as long as the plan allows; the mount check follows every swap.

Series Z: step ladder and ramp, approached from below



1

Mount the board centered and fronto-parallel; mount check; every pose of the series backs off 2 mm and moves up onto its pose

2

Step ladder at 400, 800 and 1600 mm: six rungs of 0.25 to 8 expected quanta, never below 0.1 mm; alternate Z0 and Z0 plus the step for 10 cycles, 10 frames per visit

3

Ramp at all 9 stations: the board tilted about H so the true depth across its height spans 3 expected quanta, 0.4 degrees near to 7.1 degrees far; 50 frames; the tilt is in the row

4

Log the read-back pose as it is: it is the step truth; never correct it by hand

5

Optional staircase, planned later with the same seed, only if the ramp measured a quantum of at least 0.1 mm

Series C and D: areas and detection trials



C, areas

- T4 and T5, each gap, all 9 stations, centered: 30 poses of 10 frames with random offsets
- Field sub-series at 800 mm and the small gap: the four corners, 10 poses each, pulled inward where needed
- Optional open-background variant at 800 mm: back plate off, 800 mm of clear space behind the holes



D, detection trials

- One frame per pose at a fresh random offset: one trial for every feature and blank site on the plate at once
- 60 trials per feature and station; 300 at the 3 farthest stations for the 5 percent level
- Shuffled order with a logged seed; the first frame of each C pose may count as a trial if the engineer asked for it



Rules

- Never use two frames of one pose as two trials
- Post check first: `check_captures --pilot 800` on the first C frames; posts seen above the false-alarm rate are remade thinner
- Keep frame f00 of every C pose



The detection levels are the feature-and-station pairs: 27 values of subtended size, spaced by the station ratio, overlapping by half an octave.

The pose log and the manifest



What the log holds, per pose

- procedure, target_id, gap_mm, station_z_mm, field, pose_index, frames: exactly the plan row, which also names the files
- x_mm, y_mm, z_mm: the REPORTED flange position in the base frame, at least 2 decimals
- rotation_type and r1 to r9: the orientation in the controller's own form
- timestamp, sensor_temp_c, air_temp_c



Rotation forms the tools accept

- quaternion_wxyz (ABB) or quaternion_xyzw
- euler_zyx_deg: KUKA A, B, C
- fixed_xyz_deg: FANUC W, P, R
- rotnvec_deg: Universal Robots, in degrees
- matrix: nine numbers, row by row
- make_manifest converts them all and names every file or pose that does not match



C, T5, 15, 800, 0, 17, 10, 812.40, -33.21, 455.02, euler_zyx_deg, -91.3, 19.8, 0.4,

Check each series while the targets are still set up



Low valid fraction

below half of the target's pixels

Exposure, or the target was outside the field



Outline crossing the border

a disk, cutout or square cut off

The pose is unusable; ask the engineer to re-plan it slightly inward



Target not in the field

registered target absent

The manifest or the registration is wrong for that pose



Plane fit or plane position

residual too large; plane far from the registered one

A wrongly classified plane, a wrong pose, a wrong registration or a target that moved; the limits are loose on purpose



Normal disagrees

more than 2 degrees

A tilted mount or a wrong pose

check_captures --session <folder> --out check.json prints one line per pose and a verdict; exit code 0 means nothing flagged. Re-capture flagged poses under a new pose_index; never delete lines from the log.

Things that spoil a session



Moving or bumping the sensor: everything after it is a new session



Changing exposure, gain, any filter, the base frame or the lighting mid-session



Logging the commanded pose instead of the reported one, or rounding it to 0.1 mm: for series Z the read-back pose is the step truth



Approaching a series Z pose from above or from different directions: backlash then enters the difference between visits



A loose adapter, a loosened standoff or a target shifted on its spigot: do the mount check after every swap; never swap a target to take a sentinel



Fingerprints, dust or gloss on a target, or different finishes on front and back plates: wipe with isopropyl alcohol



Using two frames of one pose as two detection trials; skipping the warm-up or shortening the settle wait; losing the plan's seed

What to hand over at the end of the session

- ✓ Characterization_<YYYYMMDD>/ with every .mc file in its series sub-folder, named by the rule of section 5
- ✓ sensor_config.json with every filter setting and the dagger values filled in; registration.json, accepted, with its residual in session_log.md
- ✓ targets_asbuilt.csv with the uncertainties; poses.csv, plan_summary.txt, plan.png, targets.json and parameters.json of the plan used, and the seed
- ✓ The pose log, and manifest.csv produced by make_manifest without errors; check.json with no flags, or a note on each
- ✓ environment_log.csv and the temperature logs; the drift run's 241 captures in sentinels/ if it was made
- ✓ session_log.md: date, serial number, warm-up and settle times, base frame, plate flatness and finish, mount-check results, target changes with times, anything unusual
- ✓ Photos of the sensor stand and of each target on the adapter (a phone camera is fine)



Software

- plan_stations, make_manifest, check_captures, register and analyze in sensorperf.cli
- Install and self-test: procedure appendix C; every tool prints its options with --help, quoted in appendix B
- The engineer runs analyze on the finished folder; results go into analysis/